

UNIVERZA V LJUBLJANI
FAKULTETA ZA RAČUNALNIŠTVO IN INFORMATIKO

Žan Vencelj

Razvoj aplikacije za upravljanje delovnega časa in kadrovskih procesov

DIPLOMSKO DELO

VISOKOŠOLSKI STROKOVNI ŠTUDIJSKI PROGRAM
PRVE STOPNJE

MENTOR: doc. dr. Aleš Smrdel

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.



Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani <http://www.gnu.org/licenses/>.

Besedilo je oblikovano z urejevalnikom besedil $\text{\LaTeX}$.

Kandidat: Žan Vencelj

Naslov: Razvoj aplikacije za upravljanje delovnega časa in kadrovskih procesov

Vrsta naloge: Diplomsko delo visokošolskega strokovnega študijskega programa prve stopnje Računalništvo in informatika

Mentor: doc. dr. Aleš Smrdel

Opis:

Pomemben dejavnik uspešnega poslovanja podjetij je učinkovito upravljanje s človeškimi viri, kar velja tako za velika kot tudi za srednja in mala podjetja. Na trgu je na voljo vrsta komercialnih rešitev za podporo procesom upravljanja s človeškimi viri, vendar so te pogosto prilagojene predvsem potrebam večjih organizacij. Namen diplomskega dela je zato razviti rešitev za upravljanje s človeškimi viri, ki bo prilagojena potrebam in značilnostim manjših podjetij. V okviru dela najprej preučite obstoječe rešitve na trgu ter na podlagi analize opredelite ključne funkcionalne in nefunkcionalne zahteve sistema. Nato načrtujte podatkovni model in arhitekturo aplikacije ter na podlagi ugotovljenih zahtev izberite ustrezne tehnologije za njeno izvedbo. Z izbranimi tehnologijami nato implementirajte opredeljene funkcionalnosti. Razviti sistem ustrezno testirajte ter predstavite in ovrednotite rezultate testiranja.

Title: Development of an Application for Work Time and Human Resources Management

Description:

An important factor in successful business operations is effective human resource management, which applies to large as well as small and medium-sized enterprises. Although a wide range of commercial solutions is available on the market to support human resource management processes, these are often tailored primarily to the needs of larger organizations. The aim of

this thesis is therefore to develop a human resource management solution tailored to the specific needs and characteristics of smaller companies. Within the scope of the thesis, first examine existing market solutions and, based on this analysis, define the key functional and non-functional requirements for the system. Next, design the data model and application architecture, and select the appropriate technologies for implementation based on the identified requirements. Using the chosen technologies, implement the defined functionalities. Finally, test the developed system thoroughly, and present and evaluate the test results.

Kazalo

Povzetek

Abstract

1	Uvod	1
1.1	Cilji naloge	2
1.2	Hipoteze	3
1.3	Struktura diplome	3
2	Pregled področja in sorodnih del	5
2.1	Komercialne rešitve	5
2.2	Razporejanje osebja	7
2.3	Dostop na osnovi vlog	8
2.4	Vrednotenje zaznane uporabnosti	8
3	Načrtovanje	9
3.1	Zajemanje zahtev	9
3.2	Podatkovni model	11
3.3	Arhitektura sistema	16
4	Tehnologije	19
4.1	Organizacija projekta	19
4.2	Zaledje	19
4.3	Spletna odjemalca	20
4.4	Mobilna aplikacija	20

4.5	Podatkovna baza in infrastruktura	21
4.6	Namestitev v produkcijsko okolje	21
5	Predstavitev sistema	23
5.1	Avtentikacija in nadzor dostopa	24
5.2	Upravljanje zaposlenih	25
5.3	Evidenca delovnega časa	26
5.4	Razporejanje izmen	26
5.5	Upravljanje odsotnosti	29
5.6	Evidenca obiskov	30
5.7	Administracija platforme	31
6	Evalvacija	33
6.1	Udeleženci in postopek	33
6.2	Vprašalnik SUS	34
6.3	Rezultati	36
7	Zaključek	39
7.1	Omejitve rešitve	40
7.2	Nadaljnje delo	40
	Literatura	43

Povzetek

Naslov: Razvoj aplikacije za upravljanje delovnega časa in kadrovskih procesov

Avtor: Žan Vencelj

V diplomskem delu je predstavljena zasnova in razvoj spletne ter mobilne aplikacije za upravljanje delovnega časa in kadrovskih procesov. Sistem združuje evidenco delovnega časa, razporejanje izmen in upravljanje odsotnosti v enotni platformi, dostopni prek spletnega in mobilnega vmesnika. Platforma ločuje pravice dostopa glede na vlogo uporabnika: administrator in kadrovska služba upravljata zaposlene, odsotnosti, urnike ter nastavitve podjetja, vodja odobrava odsotnosti in razporeja izmene svoje ekipe, zaposleni pa ima dostop do lastnega urnika, evidence delovnega časa in oddaje zahtevkov za odsotnost. Predstavljeni so zajete zahteve, podatkovni model, arhitektura sistema, uporabljene tehnologije in delovanje posameznih modulov. Poseben poudarek je namenjen modulu za samodejno generiranje osnutka urnika, ki pri razporejanju izmen upošteva razpoložljivost, preference in delovnopravne omejitve zaposlenih. Na koncu so predstavljeni še rezultati testiranja razvitega sistema: testiranje z 12 testnimi uporabniki po standardiziranem vprašalniku System Usability Scale (SUS) je pokazalo povprečno oceno 88,1 od 100, kar sistem uvršča v razred uporabnosti "odlično". Rezultati potrjujejo, da uporabniki razvito rešitev ocenjujejo kot uporabno in enostavno za uporabo.

Ključne besede: upravljanje delovnega časa, kadrovski procesi, razporejanje izmen, dostop na osnovi vlog, spletna aplikacija, mobilna aplikacija.

Abstract

Title: Development of an Application for Work Time and Human Resources Management

Author: Žan Vencelj

This thesis presents the design and development of a web and mobile application for work time and human resources management. The system combines time tracking, shift scheduling and leave management within a single platform, accessible through web and mobile interfaces. The platform separates access rights by user role: the administrator and human resources staff manage employees, leave, schedules and company settings, the manager approves leave requests and schedules shifts for their team, and the employee has self-service access to their own schedule, time records and leave requests. We describe the gathered requirements, data model, system architecture and technologies used, and present the operation of the individual modules. Particular emphasis is placed on the module for automatic draft schedule generation, which takes employee availability, preferences and labour-law constraints into account when scheduling shifts. Finally, we present the results of testing the developed system: testing with 12 test users using the standardized System Usability Scale (SUS) questionnaire yielded an average score of 88.1 out of 100, placing the system in the “excellent” usability category. The results confirm that users rate the developed solution as useful and easy to use, while the time saved through automatic shift scheduling is only indirectly supported, as it was not measured directly.

Keywords: work time management, human resources, shift scheduling, role-based access control, web application, mobile application.

Poglavje 1

Uvod

Upravljanje delovnega časa in kadrovskih procesov je za vsako organizacijo, ne glede na njeno velikost, ena od temeljnih administrativnih nalog. Podjetja morajo dosledno beležiti prihode in odhode zaposlenih, razporejati izmene, upravljati odsotnosti in nadomestila ter ob tem zagotavljati, da imajo posamezni sodelavci dostop le do podatkov in funkcij, ki ustrezajo njihovi vlogi. V manjših in srednje velikih podjetjih se te naloge pogosto še vedno opravljajo z razpršenim naborom orodij, na primer s preglednicami, papirnimi obrazci in ločenimi spletnimi storitvami za posamezne procese. Taka razdrobljenost povečuje možnost napak, otežuje pregled nad stanjem in zahteva veliko ročnega usklajevanja podatkov med sistemi.

Na trgu sicer obstajajo celovite platforme za upravljanje človeških virov (angl. Human Resources, HR), vendar so pogosto zasnovane za velika podjetja, cenovno manj dostopne za manjše organizacije ali pa premalo prilagodljive procesom, ki jih posamezna organizacija dejansko uporablja. Podroben pregled nekaterih obstoječih rešitev in znanstvenih del s področja razporejanja izmen podajamo v poglavju 2.

Motivacija za to diplomsko delo izhaja iz potrebe po razvoju enotne spletne in mobilne aplikacije, ki v enem sistemu združuje evidenco delovnega časa, razporejanje izmen in upravljanje odsotnosti, ob tem pa podpira dostop na osnovi vlog za administratorja, kadrovske službe, vodjo in zapo-

slenega. Poseben poudarek namenjamo samodejnemu predlaganju urnika, saj je ročno razporejanje izmen ob upoštevanju razpoložljivosti, preferenc in delovnopравnih omejitev zaposlenih dolgotrajno in podvrženo napakam.

1.1 Cilji naloge

Cilji diplomskega dela so:

1. zasnovati in razviti spletno aplikacijo za vodje in kadrovske službe ter mobilno aplikacijo za zaposlene, ki skupaj pokrivata celoten cikel upravljanja delovnega časa in kadrovskih procesov,
2. zasnovati dostop na osnovi vlog (angl. role-based access, RBAC), ki loči pristojnosti administratorja, kadrovske službe, vodje in zaposlenega,
3. omogočiti evidenco delovnega časa prek beleženja dogodkov, kot so prihod, odhod, odmor, delo od doma in službena pot, vključno z delovnim tokom za popravke že vnesenih dogodkov,
4. razviti modul za razporejanje izmen, ki poleg ročnega urejanja urnika podpira tudi samodejno generiranje osnutka urnika glede na razpoložljivost in preference zaposlenih,
5. razviti modul za upravljanje odsotnosti, ki zajema nastavljive tipe odsotnosti, evidenco stanja odsotnosti po zaposlenemu in delovni tok odobravanja,
6. razviti nadzorni vmesnik za upravljanje več organizacij na eni platformi,
7. ovrednotiti uporabnost razvite rešitve s pomočjo testnih uporabnikov in standardiziranega vprašalnika o uporabniški izkušnji.

Poleg naštetih ciljev smo se, ker souporablja isto infrastrukturo za avtentikacijo naprav in upravljanje organizacij, odločili razviti tudi dodaten kiosk modul za evidenco obiskov na recepciji organizacije. Ta modul ni bil

del prvotnega obsega niti ni podlaga za hipoteze v razdelku 1.2, zato ga v nadaljevanju obravnavamo ločeno od osrednjih kadrovskih procesov.

1.2 Hipoteze

V nalogi preverjamo naslednje hipoteze:

- H1** Enoten sistem, ki združuje evidenco delovnega časa, razporejanje izmen in upravljanje odsotnosti, zmanjša potrebo po ročnem usklajevanju podatkov med ločenimi orodji.
- H2** Samodejno generiranje osnutka urnika na podlagi razpoložljivosti, odsotnosti in delovnopравnih omejitev zaposlenim in vodjem prihrani čas v primerjavi z ročnim razporejanjem.
- H3** Uporabniki bodo uporabnost razvite aplikacije ocenili kot dobro, kar bo razvidno iz povprečnega rezultata vprašalnika System Usability Scale (SUS), višjega od 71 točk [2, 7].

1.3 Struktura diplome

V nadaljevanju poglavje 2 umesti razvito rešitev v obstoječi kontekst in predstavi sorodne komercialne in raziskovalne rešitve. Poglavje 3 opiše zajete zahteve, podatkovni model in arhitekturo sistema. Poglavje 4 na kratko predstavi uporabljene tehnologije in razloge za njihovo izbiro. Poglavje 5 podrobno predstavi delovanje sistema po posameznih modulih. Poglavje 6 opiše evalvacijo s testnimi uporabniki in dobljene rezultate. Poglavje 7 pa povzame dosežene cilje, kritično presodi opravljeno delo in nakaže smeri za nadaljnji razvoj.

Poglavje 2

Pregled področja in sorodnih del

V tem poglavju umestimo našo rešitev v obstoječi kontekst. Najprej predstavimo nekaj uveljavljenih komercialnih rešitev za upravljanje kadrovskih procesov, nato pa pregledamo raziskovalno literaturo s področij, na katera se pri načrtovanju sistema neposredno naslanjamo: razporejanje osebja, dostop na osnovi vlog in vrednotenje uporabnosti.

2.1 Komercialne rešitve

Na trgu obstaja več uveljavljenih platform za upravljanje kadrovskih procesov. **BambooHR** [1] je celovit kadrovski informacijski sistem, osredotočen predvsem na ameriški trg in ameriško delovnopravno zakonodajo, ki poleg evidence zaposlenih ponuja tudi orodja za ocenjevanje uspešnosti in obračun plač. **Personio** [10] sledi podobnemu konceptu, vendar je zasnovan za evropske trge in evropsko delovnopravno regulativo. **Deputy** [4] in **When I Work** [13] se v primerjavi z BambooHR in Personiem bolj osredotočata na razporejanje izmen in evidenco delovnega časa za urne delavce, na primer v gostinstvu in trgovini.

Naštete rešitve se med seboj razlikujejo tudi po tem, katere od obravnavanih funkcionalnosti dejansko pokrivajo. Vse štiri ponujajo evidenco delovnega časa in upravljanje odsotnosti, saj gre za osnovni gradnik vsake kadrovske reši-

tve. Pri razporejanju izmen je razlika večja: BambooHR je zmožnost ročnega razporejanja izmen dodal kot ločen modul Time & Attendance [1], Personio lastnega modula za razporejanje izmen nima, temveč samodejno generiranje urnika ponuja posredno, preko integracije z ločeno storitvijo Shiftbase [10], medtem ko sta Deputy in When I Work razporejanju izmen namenjena že od začetka, poleg ročnega urejanja pa oba ponujata tudi samodejno generiranje urnika glede na razpoložljivost zaposlenih [4, 13]. Po drugi strani sta BambooHR in Personio celovitejša kadrovska informacijska sistema s širšo evidenco zaposlenih (kadrovski profili, organizacijska struktura, obračun plač), medtem ko sta Deputy in When I Work osredotočena predvsem na razporejanje in evidenco delovnega časa za urne delavce, širšo kadrovsko evidenco pa ponujata le v omejenem obsegu. Tabela 2.1 povzame primerjavo funkcionalnosti.

Funkcionalnost	BambooHR	Personio	Deputy	When I Work	Naša rešitev
Evidenca delovnega časa	✓	✓	✓	✓	✓
Razporejanje izmen (ročno)	✓	✓	✓	✓	✓
Samodejno generiranje osnutka urnika	–	<i>prek integracije</i>	✓	✓	✓
Upravljanje odsotnosti	✓	✓	✓	✓	✓
Širša evidenca zaposlenih (kadrovski profil)	✓	✓	<i>delno</i>	<i>delno</i>	✓
Vsi navedeni moduli v osnovnem paketu, brez doplačila	<i>delno</i>	–	✓	✓	✓

Tabela 2.1: Primerjava funkcionalnosti razvite rešitve z izbranimi komercialnimi rešitvami.

Kot prikazuje tabela, noben od pregledanih izdelkov ne združuje evidence delovnega časa, samodejnega razporejanja izmen, upravljanja odsotnosti in širše evidence zaposlenih v enem samem sistemu: BambooHR in Personio te procese ponujata kot ločene, dodatno plačljive module, Deputy in When I Work pa jih sicer združujeta, a brez celovitejšega kadrovskega modula. Poleg tega gre pri vseh štirih za naročniške storitve v oblaku, pri katerih organizacija nima vpogleda v izvirno kodo niti možnosti prilagoditve podatkovnega modela ali poslovne logike lastnim potrebam. Naša rešitev se od naštetih razlikuje

po tem, da je zasnovana kot enoten, odprt sistem, v katerem so vsi štirje procesi od začetka povezani na skupnem podatkovnem modelu, poleg tega pa vključuje tudi nadzorni vmesnik, za upravljanje več organizacij na eni platformi.

2.2 Razporejanje osebja

Razporejanje osebja (angl. *staff scheduling*, *personnel scheduling*) je v operacijskih raziskavah dobro poznan problem. Ernst in sodelavci [5] v pregledni razpravi razporejanje osebja razdelijo na več podproblemov, na primer določanje potreb po kadrih, dodeljevanje izmen posameznikom in sestavljanje ciklov dela in počitka, ter podajo pregled metod, s katerimi jih rešujemo, od eksaktnih metod, kot je celoštevilsko programiranje, do hevrističnih in metahevrističnih pristopov, na primer genetskih algoritmov in požrešnih hevristik. Van den Bergh in sodelavci [12] v novejšem pregledu literature izpostavijo, da mora realen model razporejanja upoštevati povpraševanje po kadrih, prilagodljivo umeščanje odmorov, nadurno delo in preference zaposlenih, sicer poenostavi problem do te mere, da rešitev v praksi ni uporabna. Podobno tudi novejšo delo Liuja in sodelavcev [8] problem razporejanja osebja obravnava kot NP-težek in predlaga hitro hevristično metodo, ki poišče dovolj dobro rešitev v razumnem času, namesto da bi iskala dokazano optimalno rešitev.

Pri načrtovanju modula za samodejno razporejanje (poglavje 3) se naslanjamo prav na to zadnjo smer literature: namesto eksaktnega ali metahevrističnega reševalnika, ki bi za vsak urnik iskal optimalno rešitev, uporabimo požrešno (angl. *greedy*) hevristiko, ki izmene dodeljuje zaposlenim po vrstnem redu ob upoštevanju trdih omejitev, na primer odsotnosti, razpoložljivosti, prekrivanja izmen, tedenske omejitve ur in minimalnega počitka med izmenama, ter mehkih kriterijev, na primer preferenc zaposlenih in enakomerne porazdelitve ur med zaposlenimi. Tak pristop je hitrejši in enostavnejši za razumevanje in vzdrževanje kot eksaktni ali metahevristični pristopi, cena za to pa je, da dobljeni urnik ni nujno optimalen, temveč zgolj dovolj dober

izhodiščni osnutek, ki ga vodja lahko še ročno popravi.

2.3 Dostop na osnovi vlog

Dostop na osnovi uporabniških vlog je model za upravljanje pravic dostopa, pri katerem pravice niso dodeljene neposredno posameznim uporabnikom, temveč vlogam, uporabniki pa so tem vlogam nato dodeljeni. Sandhu in sodelavci [11] so v temeljnem delu na tem področju predstavili družino referenčnih modelov RBAC, od osnovnega modela, ki povezuje uporabnike, vloge in pravice, do razširitev s hierarhijo vlog in ločevanjem dolžnosti. V razviti aplikaciji uporabimo poenostavljeno različico osnovnega modela s štirimi fiksnimi vlogami, administratorjem, kadrovsko službo, vodjo in zaposlenim, pri čemer nimamo hierarhije vlog, kar podrobneje opišemo v poglavjih 3 in 5.

2.4 Vrednotenje zaznane uporabnosti

Za vrednotenje zaznane uporabnosti poslovnih aplikacij sta uveljavljena predvsem dva kratka standardizirana vprašalnika. *System Usability Scale* (SUS), ki ga je predlagal Brooke [3]. Vsebuje deset trditev, na katere uporabnik odgovarja s petstopenjsko lestvico strinjanja (Likertovo lestvico), odgovori pa se pretvorijo v skupno oceno med 0 in 100. Finstad [6] je pozneje predlagal krajši, štiritrditveni vprašalnik *Usability Metrics For User Experience* (UMUX), ki naj bi dajal primerljive rezultate kot SUS, a je hitrejši za izpolnjevanje. Ker gre za standardizirana in v raziskovalni literaturi pogosto uporabljena vprašalnika, ju uporabimo tudi pri vrednotenju razvite rešitve v poglavju 6.

Poglavje 3

Načrtovanje

V tem poglavju so opisane zajete zahteve, podatkovni model in arhitektura razvite aplikacije. Odločitve, predstavljene v tem poglavju, so podlaga za izbiro tehnologij v poglavju 4 in za opis delovanja sistema v poglavju 5.

3.1 Zajemanje zahtev

Zahteve smo zajeli na podlagi lastne analize tipičnih kadrovskih procesov v manjših in srednje velikih podjetjih ter pregleda funkcionalnosti obstoječih komercialnih rešitev, predstavljenih v poglavju 2.1. Ker diplomsko delo ne vključuje formalnega naročnika, smo si pri opredelitvi zahtev pomagali z vlogami, ki v realni organizaciji sodelujejo pri kadrovskih procesih: administrator platforme, kadrovska služba, vodja tima in zaposleni. Zahteve smo med razvojem večkrat dopolnili, saj se je izkazalo, na primer pri razporejanju izmen, da posamezne poenostavitve (na primer, da ima zaposleni le binarno razpoložljivost, torej da/ne) ne zadostujejo za realistično uporabo.

3.1.1 Funkcionalne zahteve

Funkcionalne zahteve smo razvrstili glede na vlogo uporabnika.

Administrator upravlja organizacije na platformi (ustvarjanje, onemogo-

čanje, brisanje), pregleduje uporabnike in revizijsko sled sprememb ne glede na organizacijo, ne posega pa v vsakodnevne kadrovske procese posamezne organizacije.

Kadrovska služba upravlja zaposlene (vabila, profili, vloge), nastavlja tipe odsotnosti in njihova pravila ter potrjuje ali zavrača zahteve za odsotnost.

Vodja upravlja urnik svojega tima: ročno ustvarja in ureja izmene, ustvarja ponavljajoče se izmene in predloge zasedenosti, sproži samodejno generiranje osnutka urnika, ga po potrebi ročno popravi in ga objavi. Vodja tudi potrjuje ali zavrača zahteve za popravek dogodkov delovnega časa.

Zaposleni beleži dogodke delovnega časa (prihod, odhod, odmor, delo od doma, službena pot), oddaja zahteve za popravek že vnesenih dogodkov, nastavlja svojo tedensko razpoložljivost in preference, prevzema odprte izmene ter oddaja zahteve za odsotnost.

Zahteve, skupne več vlogam, so še: prijava v sistem z geslom in preverjanjem elektronskega naslova, pregled lastne zgodovine dogodkov, izmen in odsotnosti ter prejemanje obvestil o pomembnih dogodkih (na primer objava urnika).

3.1.2 Nefunkcionalne zahteve

- **Podpora več organizacijam:** sistem mora na eni namestitvi podpirati več neodvisnih organizacij, pri čemer podatki ene organizacije ne smejo biti vidni drugi (t.i. *multi-tenancy*). Tovrstno ločevanje podatkov med strankami je standardna arhitekturna praksa vseh v poglavju 2.1 obravnavanih oblačnih rešitev.
- **Nadzor dostopa:** vsak API klic mora preveriti tako pristnost uporabnika kot tudi njegovo vlogo, preden dovoli dostop do posameznega vira.

- **Dostopnost na več napravah:** vodje in kadrovska služba delo opravljajo v spletnem brskalniku, zaposleni pa dogodke delovnega časa najpogosteje beležijo prek mobilne naprave, zato mora sistem ponuditi tako spletno kot mobilno aplikacijo.
- **Odzivno delovanje:** operacije, ki niso nujno sinhrono (na primer pošiljanje e-pošte in potisnih obvestil), naj ne blokirajo odziva na uporabnikovo zahtevo.
- **Sledljivost:** spremembe, ki jih izvede administrator platforme, morajo biti zabeležene v revizijski sledi.

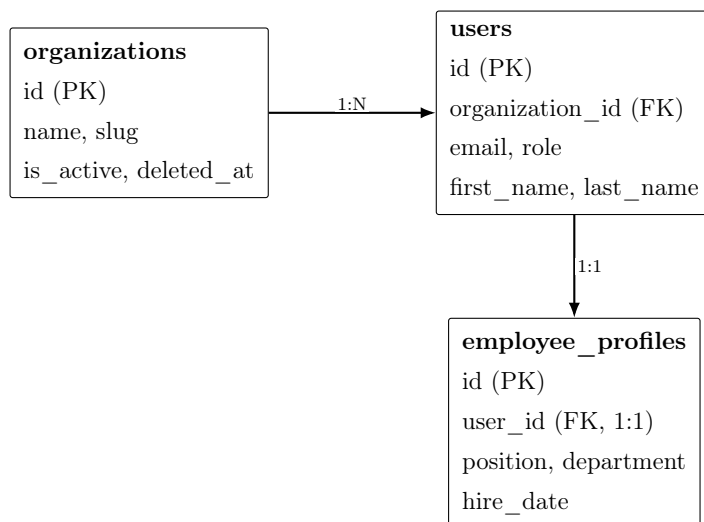
3.2 Podatkovni model

Odločili smo se za relacijski podatkovni model. Skoraj vse tabele, ki pripadajo posamezni organizaciji, vsebujejo tuji ključ `organization_id`, kar omogoča ločevanje podatkov med organizacijami na ravni posamezne vrstice (t.i. vzorec *row-level multi-tenancy*). Izjema je tabela `organizations` sama ter tabela `admin_audit_logs`, ki jo uporablja izključno administrator platforme in presega meje posamezne organizacije. Zaradi preglednosti podatkovni model predstavimo po posameznih domenah: identiteta in organizacije (slika 3.1), evidenca delovnega časa (slika 3.2), razporejanje izmen (slika 3.3), upravljanje odsotnosti (slika 3.4) in podatkovni model dodatnega kiosk modula za evidenco obiskov (slika 3.5). Poleg prikazanih entitet podatkovna baza vsebuje še podporne tabele za seje, vabila, ponastavitve gesla in preverjanje elektronske pošte, ki jih zaradi preglednosti v diagramih ne prikazujemo posebej.

3.2.1 Identiteta in organizacije

Slika 3.1 prikazuje osrednje entitete za upravljanje identitete. Vsak uporabnik (tabela `users`, v nadaljevanju podpoglavja so v oklepajih navedene tabele) pripada največ eni organizaciji (`organizations`) in ima natanko eno izmed vlog: `admin`, `hr`, `manager`, `employee` ali `superadmin`. Slednjo ima izključno

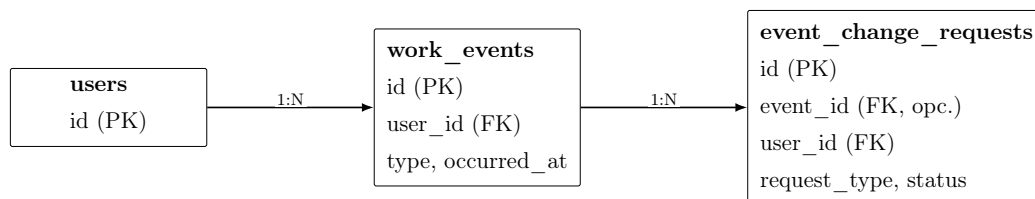
administrator platforme, ki organizaciji ne pripada. Uporabnik z vlogo **employee** ima dodaten profil (**employee_profiles**) s kadrovskimi podatki, na primer delovnim mestom, oddelkom in datumom zaposlitve.



Slika 3.1: Poenostavljen podatkovni model identitete in organizacij.

3.2.2 Evidenca delovnega časa

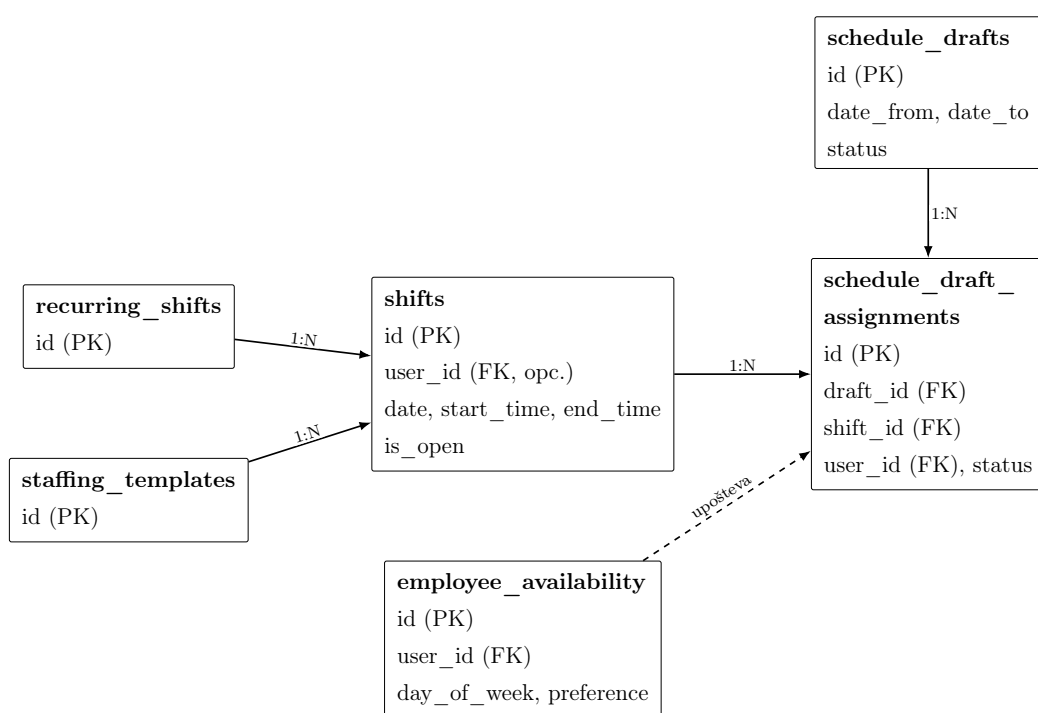
Slika 3.2 prikazuje del podatkovnega modela za evidenco časa. Zaposleni dogodke delovnega časa beleži kot vrstice v tabeli **work_events**, pri čemer tip dogodka (prihod, odhod, začetek in konec odmora, delo od doma, začetek in konec službene poti) določa nabor **work_event_type**. Ker se dogodki beležijo v realnem času in napake pri vnosu niso redke, ima zaposleni možnost oddati zahtevek za popravek (**event_change_requests**), ki lahko obstoječ dogodek doda, uredi ali izbriše. Zahtevek pa potrdi ali zavrne vodja.



Slika 3.2: Podatkovni model evidence delovnega časa in zahtevkov za popravek.

3.2.3 Razporejanje izmen

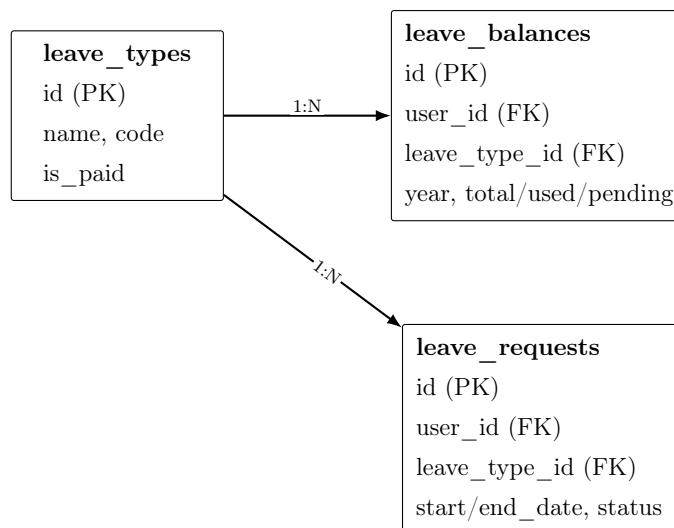
Osrednja entiteta razporejanja je izmena (**shifts**), ki je vezana na organizacijo, opcijsko pa tudi na uporabnika, ki jo bo opravil, izmena brez dodeljenega uporabnika je odprta izmena, ki jo lahko prevzame kateri koli ustrezen zaposleni. Izmena lahko nastane ročno, iz ponavljajočega se vzorca (**recurring_shifts**) ali iz predloge zasedenosti (**staffing_templates**), ki za posamezen dan v tednu določa željeno število zaposlenih. Pri samodejnem razporejanju vodja ustvari osnutek urnika (**schedule_drafts**) za izbrano obdobje, reševalnik nato za vsako izmeno v tem obdobju predlaga dodelitev (**schedule_draft_assignments**) ob upoštevanju razpoložljivosti in preferenc zaposlenega (**employee_availability**). Ta del podatkovnega modela prikazuje slika 3.3, delovanje reševalnika pa podrobneje opišemo v poglavju 5.



Slika 3.3: Podatkovni model razporejanja izmen in samodejnega generiranja urnika.

3.2.4 Upravljanje odsotnosti

Del podatkovnega modela, ki se nanaša na odsotnost je prikazan na sliki 3.4. Vsaka organizacija si nastavi lastne tipe odsotnosti (**leave_types**), na primer letni dopust ali bolniška odsotnost, vsakemu pa določi, ali je plačan, in privzeto letno kvoto dni. Za vsakega zaposlenega in vsak tip odsotnosti se po letih vodi stanje (**leave_balances**) s skupnim, izrabljenim in čakajočim številom dni. Zahtevek za odsotnost (**leave_requests**) gre skozi delovni tok od oddaje do odobritve ali zavrnitve.

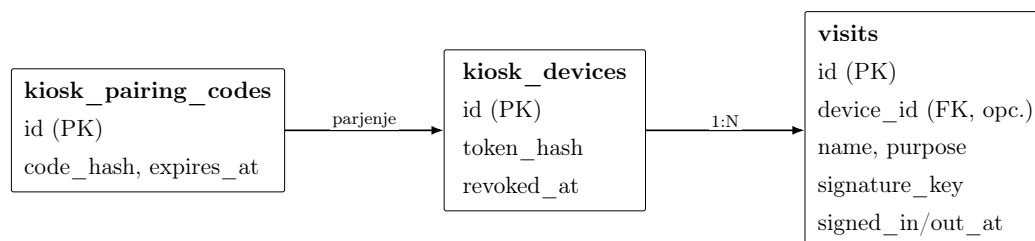


Slika 3.4: Podatkovni model upravljanja odsotnosti.

3.2.5 Evidenca obiskov

Ta del podatkovnega modela prikazuje slika 3.5. Dodatni kiosk modul, opisan v razdelku 5.6, potrebuje lasten, majhen del podatkovnega modela. Tablica na recepciji se organizaciji priklapi (spari) s kodo za paritev (**kiosk_pairing_codes**), ki jo ustvari administrator organizacije in velja omejen čas. Ob uspešnem parjenju zaledje ustvari trajen zapis naprave (**kiosk_devices**) z zgoščenim žetonom za dostop naprave. Vsak obisk (**visits**) je vezan na organizacijo in napravo, prek katere je bil vnesen. Hrani

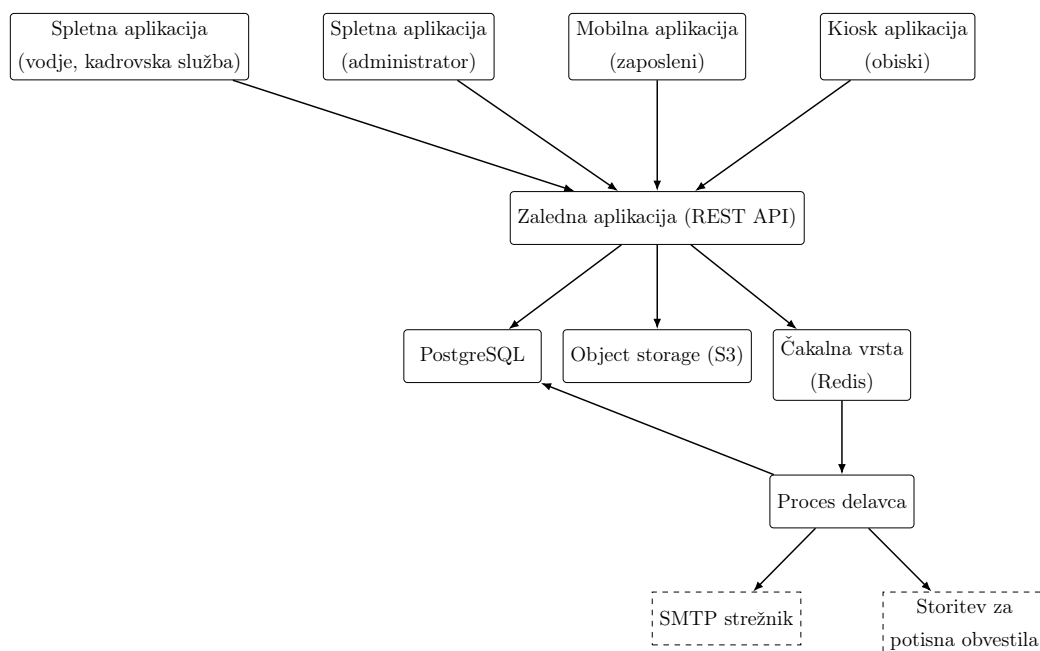
pa ime in namen obiska ter sklic na podpis, shranjen v objektno shrambo (angl. object storage).



Slika 3.5: Podatkovni model evidence obiskov.

3.3 Arhitektura sistema

Sistem je zasnovan po arhitekturi odjemalec-strežnik. Zaledje je ena sama aplikacija REST, ki jo uporabljajo štiri odjemalske aplikacije: spletna aplikacija za vodje in kadrovske službo, mobilna aplikacija za zaposlene, ločena spletna aplikacija za administratorja platforme ter kiosk aplikacija na tablici za evidenco obiskov. Vsi zapisi domenskih podatkov so shranjeni v skupni podatkovni bazi PostgreSQL, do katere odjemalci dostopajo izključno prek zaledne aplikacije. Slike podpisov obiskovalcev zaledje namesto v bazo shrani v objektno shrambo, združljivo z vmesnikom S3. Opravila, ki jih ni treba obdelati sinhrono med spletno zahtevo, na primer pošiljanje e-pošte ali potisnih obvestil, zaledje odloži v čakalno vrsto, ki jo obdeluje ločen proces delavca (angl. *worker*). Slika 3.6 prikazuje visokonivojsko arhitekturo sistema.



Slika 3.6: Visokonivojska arhitektura sistema.

Znotraj zaledne aplikacije je koda razdeljena na module po domenah, na primer avtentikacija, zaposleni, izmene, razporejanje, odsotnosti, dogodki in administracija, kar je podrobneje opisano v poglavju 5. Vsak dostop do zaščenega vira zaledje preveri v dveh korakih: najprej preveri veljavnost dostopnega žetona, nato pa še, ali vloga uporabnika ustreza vlogam, ki jih posamezen vir dovoljuje. Ta mehanizem je podrobneje opisan v poglavju 5 skupaj z ostalimi vidiki delovanja sistema.

Poglavje 4

Tehnologije

V tem poglavju na kratko naštejemo in utemeljimo izbiro tehnologij, s katerimi smo zgradili sistem, opisan v poglavju 3.

4.1 Organizacija projekta

Celoten projekt je organiziran kot enoten repozitorij (t. i. *monorepo*), upravljan z orodjem Nx. Nx omogoča, da zaledje, oba spletna odjemalca in mobilno aplikacijo vzdržujemo v enem repozitoriju, si delijo skupne knjižnice (na primer tipe podatkov in validacijske sheme) in imajo enotno orodje za gradnjo, testiranje in preverjanje odvisnosti med moduli. Za upravljanje paketov uporabljamo pnpm, za tipski nadzor pa TypeScript v celotnem projektu, tako na zaledju kot na obeh vrstah odjemalcev.

4.2 Zaledje

Zaledna aplikacija je zgrajena na ogrodju NestJS, ki nad Node.js ponuja modularno strukturo z vbrizgavanjem odvisnosti (angl. *dependency injection*), kar omogoča ločevanje kode po domenskih modulih. Za dostop do podatkovne baze PostgreSQL uporabljamo Drizzle ORM, ki poizvedbe tipsko preveri že v času prevajanja in migracije sheme upravlja s samostojnimi datotekami SQL,

kar omogoča natančen nadzor nad spremembami sheme. Za odložena opravila, na primer pošiljanje e-pošte in potisnih obvestil, uporabljamo knjižnico BullMQ, ki čakalno vrsto hrani v pomnilniški podatkovni bazi Redis in jo v ločenem procesu delavca obdeluje zunaj glavne zahteve. Gesla zaposlenih hranimo zgoščena z algoritmom Argon2, za avtentikacijo pa uporabljamo dostopne in osvežitvene žetone JWT.

4.3 Spletna odjemalca

Spletni aplikaciji za vodje in kadrovske službe ter za administratorja platforme sta zgrajeni s knjižnico React in orodjem Vite. Za usmerjanje in pridobivanje podatkov s strežnika uporabljamo TanStack Router in TanStack Query, ki poskrbita za predpomnjenje, ponovno pridobivanje in usklajevanje stanja s strežnikom, ne da bi za to potrebovali ločen globalni shranjevalnik stanja za vsak posamezen vir podatkov. Za manjše količine lokalnega stanja uporabniškega vmesnika, ki ni neposredno vezano na strežniške podatke, uporabimo Zustand. Za oblikovanje uporabljamo ogrodje Tailwind CSS, za preverjanje oblik in odgovorov strežnika pa knjižnico Zod.

4.4 Mobilna aplikacija

Mobilno aplikacijo za zaposlene smo zgradili z ogrodjem Expo nad React Native, kar omogoča razvoj ene kodne baze za Android in iOS. Za oblikovanje uporabniškega vmesnika uporabljamo NativeWind, ki v mobilno okolje prenese enak način stiliziranja z razredi, kot ga uporabljamo v spletnih odjemalcih s Tailwind CSS, kar olajša skupno razumevanje oblikovanja med vsemi odjemalskimi aplikacijami.

4.5 Podatkovna baza in infrastruktura

Za primarno shrambo podatkov uporabljamo relacijsko podatkovno bazo PostgreSQL. Slike podpisov obiskovalcev iz razdelka 5.6 namesto v relacijsko bazo shranjujemo v objektno shrambo, prek vmesnika S3, kar prepreči nepotrebno rast same podatkovne baze z binarno vsebino. Za razvojno okolje uporabljamo Docker Compose, ki poleg PostgreSQL zažene še Redis za čakalno vrsto, MinIO kot lokalno združljivo zamenjavo za S3 ter Mailhog, ki v razvoju prestreže odhodno e-pošto brez dejanskega pošiljanja.

4.6 Namestitev v produkcijsko okolje

Zaledno aplikacijo in oba spletna odjemalca namestimo na navidezni zasebni strežnik pri ponudniku Hetzner. Za zaledje, proces delavca in obratni posrednik (opisan spodaj) iz istega repozitorija zgradimo tri samostojne slike vsebnika Docker in jih naložimo v GitHub Container Registry (GHCR). Namestitveni strežnik nobene slike ne gradi sam, temveč jih ob vsaki namestitvi zgolj prenese in zažene z orodjem Docker Compose prek datoteke `docker-compose.prod.yml`. Slike označimo z identifikatorjem spremembe (angl. *commit hash*) v nadzoru različic Git, kar ob morebitni okvari omogoča vrnitev na predhodno različico, ne da bi bilo treba karkoli ponovno graditi.

Edini del sistema, izpostavljen internetu, je obratni posrednik (angl. *reverse proxy*) Caddy, ki samodejno pridobi in obnavlja potrdila TLS prek protokola ACME ter dohodne zahteve usmerja glede na domeno. Spletni aplikaciji za vodje in kadrovske službe ter za administratorja platforme vsaka na svoji domeni strežeta lastne statične datoteke, pot `/api/*` pa obe posredujeta isti zaledni aplikaciji. Ker gre pri tem za zahtevo z istim izvorom (angl. *same-origin*), se odjemalcema ni treba ukvarjati z omejitvami CORS. Objektna shramba iz razdelka 3.3 je izpostavljena na ločeni domeni, saj so povezave do shranjenih datotek podpisane glede na točno določen naslov gostitelja. Pred zagonom zaledne aplikacije in procesa delavca se enkratno požene še storitev za migracije podatkovne baze, ki počaka na dokončanje

vseh čakajočih migracij sheme, šele nato pa se zaženeta preostali dve storitvi. Podatkovno bazo vsako noč samodejno varnostno kopiramo z omejenim številom hranjenih kopij, delovanje celotnega sistema pa nadziramo z orodjema Grafana in Loki za beleženje in pregledovanje dnevnikov.

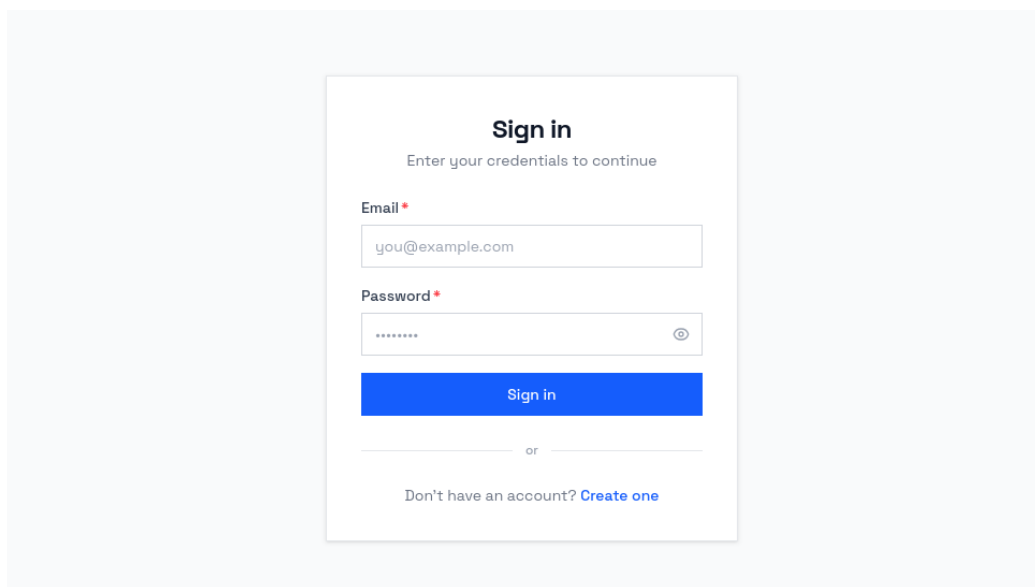
Mobilno aplikacijo za zaposlene distribuiramo ločeno od zaledja in spletnih odjemalcev, saj gradnjo za Android in iOS namesto lokalno izvedemo v oblaku prek storitve EAS Build ponudnika Expo. Gradnja za Android vrne namestitveno datoteko (APK), ki jo uporabnik namesti neposredno na napravo, gradnja za iOS pa zaradi omejitev platforme Apple zahteva veljaven razvijalski račun Apple Developer in podpisano različico, ki jo prek storitve TestFlight objavimo testnim uporabnikom. Slednji aplikacijo namestijo prek namenske aplikacije TestFlight, ne da bi bilo treba njihove naprave predhodno registrirati v našem računu Apple Developer.

Poglavje 5

Predstavitev sistema

V tem poglavju opišemo delovanje sistema po posameznih modulih: kaj se zgodi, ko uporabnik izvede določeno dejanje v odjemalski aplikaciji, kateri klic proti zaledni aplikaciji to dejanje sproži in kako se odrazi v podatkovni bazi. Zaledna koda je razdeljena na module po domenah: avtentikacija, zaposleni, izmene, razporejanje, odsotnosti, dogodki, obiski in administracija. Vsak modul izpostavi svoj nabor poti REST znotraj skupne aplikacije, opisane v razdelku 3.3.

Slika 5.1 prikazuje prijavno stran spletne aplikacije za vodje in kadrovske službe. Enak vzorec obrazca (elektronski naslov, geslo) uporabljajo tudi ostale spletne aplikacije, opisane v nadaljevanju poglavja.



Slika 5.1: Prijavna stran spletne aplikacije za vodje in kadrovske službe.

5.1 Avtentikacija in nadzor dostopa

Nova organizacija pridobi zaposlene prek povabil: kadrovska služba na naslov zaposlenega pošlje povabilo, ki ga zaledje shrani v tabelo `invitations` skupaj z zgoščeno vrednostjo enkratnega žetona. Zaposleni ob sprejemu povabila izbere geslo, zaledje pa ob prijavi preveri elektronski naslov z enkratno kodo (angl. One-Time Password, OTP), ki jo pošlje po elektronski pošti prek čakalne vrste, opisane v razdelku 3.3. Zaledje po petih zaporednih napačnih poskusih vnosa kode nadaljnje poskuse zavrne in zahteva ponoven začetek registracije, potekle ali že uporabljene kode pa zavrne takoj.

Ob uspešni prijavi zaledje izda dva žetona JWT: kratkoživ dostopni žeton, ki ga odjemalec hrani v pomnilniku in ga pošilja v glavi `Authorization`, ter dolgoživ osvežitveni žeton, ki ga zaledje shrani kot sejo (`sessions`) in odjemalcu vrne v piškotku, vezanem na napravo. Taka delitev omeji izpostavljenost dostopnega žetona, hkrati pa uporabniku ni treba gesla vnašati znova ob vsakem poteku dostopnega žetona. Ker si spletna aplikacija za vodje in administratorska aplikacija delita isto zaledno domeno, zaledje piškotka za

navadne uporabnike in administratorja loči po imenu, dostopni žeton v glavi **Authorization** pa ima vedno prednost pred piškotkom, da preprečimo, da bi zastarel piškotek ene aplikacije preglasil žeton druge.

Vsak klic na zaščiteno pot gre skozi dve zaporedni varovali. Prvo (**JwtAuthGuard**) preveri veljavnost dostopnega žetona in iz njega prebere identiteto ter vlogo uporabnika. Poti, označene kot javne, preverjanje preskočijo. Drugo (**RolesGuard**) preveri, ali vloga uporabnika ustreza vlogam, ki jih zahteva klicana pot, sicer zahtevo zavrne s stanjem 403. Tabela 5.1 povzame, katere vloge imajo dostop do posameznih sklopov funkcionalnosti.

Tabela 5.1: Poenostavljen pregled dostopa do posameznih sklopov po vlogah.

Sklop	Admin	HR	Vodja	Zaposleni	Superadmin
Lastni dogodki, izmene, odsotnosti	✓	✓	✓	✓	–
Urejanje izmen tima	✓	✓	✓	–	–
Nastavitve razporejanja	✓	✓	–	–	–
Upravljanje zaposlenih in povabil	✓	✓	✓	–	–
Potrjevanje odsotnosti in popravkov	✓	✓	✓	–	–
Organizacije čez celotno platformo	–	–	–	–	✓

5.2 Upravljanje zaposlenih

Kadrovska služba v spletni aplikaciji ustvari povabilo z elektronskim naslovom, imenom, priimkom in vlogo novega sodelavca. Zaledje ustvari zapis v tabeli **invitations** in po čakalni vrsti odloži pošiljanje elektronske pošte s povezavo do sprejema povabila. Ko zaposleni povabilo sprejme in izbere geslo, zaledje ustvari nov zapis v tabeli **users** z ustrezno vlogo in organizacijo ter pripadajoč **employee_profiles** z osnovnimi kadrovskimi podatki, povabilo pa označi kot sprejeto. Kadrovska služba lahko kadar koli pregleduje seznam zaposlenih, ureja podatke posameznega profila (delovno mesto, oddelek, najvišje tedensko število ur) ali prekliče še nesprejeto povabilo.

5.3 Evidenca delovnega časa

Zaposleni v mobilni aplikaciji ob prihodu, odhodu, začetku ali koncu odmora ter podobnih dogodkih sproži enostaven klic, ki v tabelo `work_events` doda vrstico s tipom dogodka in časovnim žigom. Ker se dogodki beležijo sproti in je napačen pritisk pogost, zaposleni namesto neposrednega urejanja že zabeleženega dogodka odda zahtevek za popravek: ta se shrani v tabelo `event_change_requests` skupaj z zahtevanim tipom dogodka, časom in razlogom, obstoječi dogodek pa ostane nespremenjen do odobritve. Vodja v spletni aplikaciji pregleduje čakajoče zahteve svojega tima. Ob odobritvi zaledje glede na vrsto zahtevka (dodaj, uredi, izbriši) ustrezno spremeni izvorni zapis v `work_events` in zahtevek označi kot odobren, ob zavrnitvi pa zgolj zabeleži razlog in izvorni dogodek pusti nespremenjen.

5.4 Razporejanje izmen

Vodja urnik lahko sestavlja ročno, izmeno po izmeno, ali s ponavljajočimi se vzorci in predlogami zasedenosti, opisanimi v razdelku 3.2. Izmena brez dodeljenega zaposlenega je označena kot odprta, kateri koli ustrezen zaposleni jo lahko v mobilni ali spletni aplikaciji prevzame, zaledje pa ob tem preveri, da se izmena ne prekriva z že dodeljenimi izmenami tega zaposlenega. Slika 5.2 prikazuje tedenski pregled urnika v spletni aplikaciji za vodje.

Employee	Mon 27 today	Tue 28	Wed 29	Thu 30	Fri 31	Sat 01	Sun 02	Hrs
Admin User								0.0h
Team Manager	09:00-17:00		09:00-17:00					16.0h
HRRRR Manager		08:00-16:00				08:00-14:00		14.0h
Alice Smith	08:00-16:00	08:00-16:00		12:00-20:00				24.0h

Slika 5.2: Tedenski pregled urnika v spletni aplikaciji za vodje.

Za samodejno sestavljanje urnika vodja izbere časovno obdobje, za katero želi predlog. Zaledje najprej iz predlog zasedenosti za to obdobje ustvari manjkajoče izmene, nato pa ustvari osnutek urnika (`schedule_drafts`) in nad njim požene reševalnik. Reševalnik izmene obravnava kronološko in za vsako izmeno med kandidati izbere najprimernejšega zaposlenega, kot prikazuje algoritem 1.

Algoritem 1 Poenostavljen potek požrešnega reševalnika razporejanja

```
1: function RAZPOREDIOSNUTEK(izmene, zaposleni, nastavitve)
2:   razvrsti izmene kronološko po datumu in času začetka
3:   stanje  $\leftarrow$  prazna preslikava (zasedeni termini in tedenske ure na zaposlenega)
4:   for vsako izmeno s v izmenah do
5:     ocene  $\leftarrow$  []
6:     for vsakega zaposlenega z v zaposlenih do
7:       izračunaj primernost z za s: odsotnosti, razpoložljivost, prekrivanje, tedenska omejitev ur, minimalni počitek
8:       ocene.dodaj((z, primeren?, preferenca, tedenske ure))
9:     end for
10:    primerni  $\leftarrow$  filtriraj ocene po primeren?
11:    if primerni ni prazen then
12:      najboljši  $\leftarrow$  tisti z najvišjo preferenco (izbrana strategija razveže neodločeno po tedenskih urah)
13:      dodeli s najboljšemu, posodobi njegovo stanje
14:    else
15:      označi s kot nezasedeno, zabeleži razlog (npr. ni razpoložljivega zaposlenega)
16:    end if
17:  end for
18:  return predlagane dodelitve
19: end function
```

Organizacija lahko izbere eno izmed treh strategij razvrščanja, ki določajo, kateremu kriteriju reševalnik pri izbiri med primernimi zaposlenimi da prednost:

- **preference_first** – reševalnik najprej izbere zaposlenega z najvišjo preferenco za dani termin, med enako preferenčnimi zaposlenimi pa tistega z manj že dodeljenimi urami v tednu.
- **fairness_first** – obrne vrstni red kriterijev: prednost dobi zaposleni z manj že dodeljenimi urami v tednu, preferenca pa odloči šele, če imata dva zaposlena enako število ur.
- **preference_only** – razvršča enako kot **preference_first**, a med primerne za dani termin uvrsti le tiste zaposlene, ki so ga izrecno označili kot preferiranega, s čimer se preferenca upošteva strogo, ne le kot kriterij razvrščanja.

Ker gre za požrešno hevrstiko in ne za eksaktni reševalnik, kot smo utemeljili v razdelku 2.2, dobljeni osnutek ni nujno optimalen: vodja ga v spletni aplikaciji lahko pregleda, posamezne dodelitve ročno spremeni in šele nato objavi. Slika 5.3 prikazuje osnutek urnika za en teden, kjer je reševalnik vseh deset izmen uspešno dodelil štirim zaposlenim, vsako izmeno enemu izmed njih. Med objavo zaledje vsem zaposlenim z dodeljeno izmeno po čakalni vrsti pošlje potisno obvestilo, če pa to ni mogoče, obvestilo pošlje po elektronski pošti.

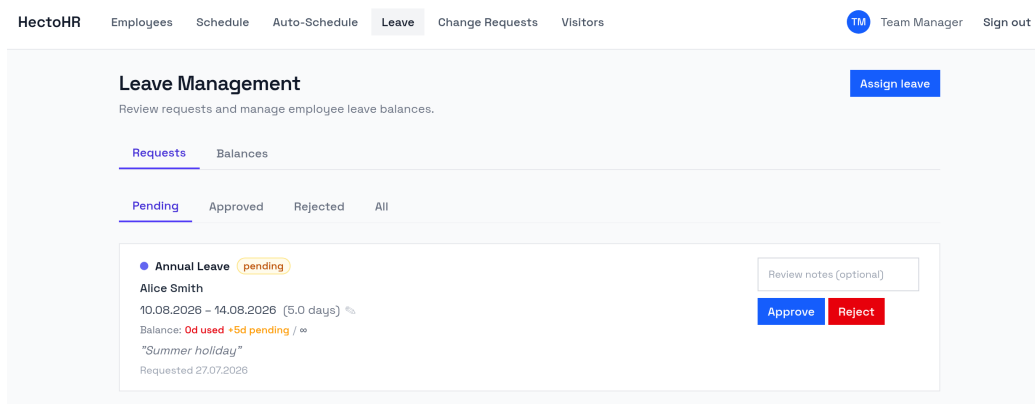
The screenshot shows the 'Auto-Schedule' interface in HectoHR. At the top, there's a navigation bar with 'HectoHR', 'Employees', 'Schedule', 'Auto-Schedule' (active), 'Leave', 'Change Requests', and 'Visitors'. On the right, there's a 'Team Manager' button and a 'Sign out' link. Below the navigation bar, the 'Auto-Schedule' section has a subtitle 'Generate a conflict-free draft roster from open shifts, availability and leave'. There are three tabs: 'Schedule builder' (active), 'Staffing templates', and 'Settings'. Under 'Schedule builder', there are two date pickers: 'From' (03. 08. 2026) and 'To' (09. 08. 2026), followed by a 'Regenerate draft' button. Below this, it says 'Draft for 2026-08-03 – 2026-08-09 · 10 assigned · 0 unfilled'. There are two buttons: 'Approve & publish' and 'Discard'. The main part of the interface is a table showing the draft roster.

Employee	Mon 03	Tue 04	Wed 05	Thu 06	Fri 07	Sat 08	Sun 09
Admin User		08:00-17:00		08:00-17:00			
Team Manager	08:00-17:00		08:00-17:00		08:00-17:00		
HRRRR Manager	08:00-17:00		08:00-17:00		08:00-17:00		
Alice Smith		08:00-17:00		08:00-17:00			
Unassigned							

Slika 5.3: Osnutek urnika, ki ga je predlagal reševalnik za samodejno razporejanje.

5.5 Upravljanje odsotnosti

Kadrovska služba za organizacijo nastavi tipe odsotnosti (na primer letni dopust, bolniška odsotnost) z lastnostmi, ali je tip plačan in koliko dni na leto pripada posameznemu zaposlenemu privzeto. Zaledje nato za vsakega zaposlenega in leto ustvari ustrezen zapis stanja odsotnosti (`leave_balances`). Zaposleni v mobilni aplikaciji odda zahtevek za odsotnost z izbranim tipom in obdobjem, zaledje pa samodejno izračuna število delovnih dni in stanje označi kot čakajoče. Vodja ali kadrovska služba zahtevek odobri ali zavrne. Ob odobritvi zaledje čakajoče dni prenese med izrabljene, ob zavrnitvi pa jih vrne nazaj v razpoložljivo stanje. Kadrovska služba lahko dneve v stanju odsotnosti tudi ročno popravi, na primer ob prenosu neizrabljenih dni iz preteklega leta. Slika 5.4 prikazuje pregled čakajočih zahtevkov za odsotnost v spletni aplikaciji za vodje, skupaj z gumboma za odobritev in zavrnitev.



Slika 5.4: Pregled čakajočih zahtevkov za odsotnost.

5.6 Evidenca obiskov

Poleg osrednjih kadrovskih procesov sistem vključuje še dodaten, samostojen modul za evidenco obiskov na recepciji organizacije, ki smo ga omenili že v uvodu (razdelek 1.1). Modul teče kot mobilna aplikacija na poljubni napravi na recepciji, najpogosteje tablici, ki jo je treba pred uporabo z zaledjem enkratno seznaniti (parjenje), s čimer vzpostavi lastno zaupanje do zaledja, neodvisno od uporabniške prijave.

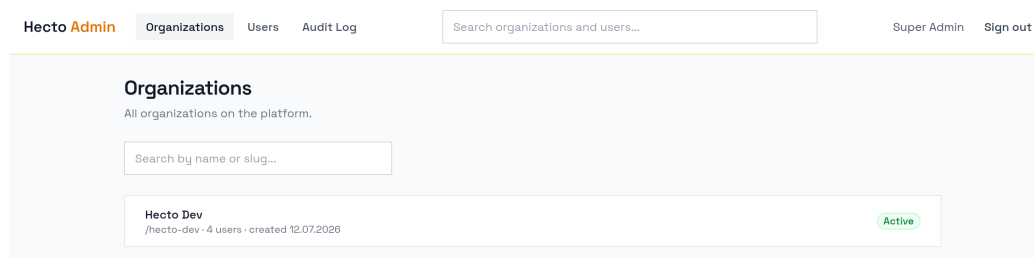
Administrator organizacije v spletni aplikaciji ustvari paritveno kodo z omejeno veljavnostjo in imenom naprave (na primer “Recepcija”), to kodo pa nato enkratno vnese v kiosk aplikacijo na tablici. Zaledje ob uspešnem parjenju kodo porabi in tablici izda dolgoživ žeton naprave, ki ga kiosk aplikacija odslej pošilja namesto uporabniškega žetona JWT. Poti `/kiosk/*` so zato v celoti izvzete iz varoval, opisanih v razdelku 5.1, in namesto tega preverjajo izključno veljavnost žetona naprave. S tem se izognemo temu, da bi moral vsak obiskovalec za enkratni obisk imeti svoj uporabniški račun.

Obiskovalec na tablici vnese ime in namen obiska ter se podpiše na zaslon, kiosk aplikacija podpis kot sliko naloži neposredno v objektno shrambo, opisano v razdelku 3.3, zaledje pa ustvari zapis obiska (tabela `visits`) s časom prihoda in sklicem na naloženo sliko. Recepcija obiskovalca ob odhodu na tablici označi kot odjavljenega, če pa to pozabi, zaledje obisk samodejno

zapre ob koncu dne. Zaposleni z ustrezno vlogo lahko v spletni aplikaciji kadar koli preverijo, kdo je trenutno v prostorih organizacije, in pregledajo pretekle obiske.

5.7 Administracija platforme

Administrator platforme (vloga **superadmin**) do sistema dostopa prek ločenega prijavnega mesta, ki sprejme le uporabnike s to vlogo. Ta uporabnik ne pripada nobeni organizaciji in ni ustvarjen prek običajnega vabila, temveč izključno prek namenskega ukaza ob namestitvi sistema. Administratorska aplikacija administratorju omogoča iskanje in pregled organizacij, kar prikazuje slika 5.5, ter uporabnikov čez celotno platformo, onemogočanje ali izbris organizacije, ki posledično prijavo uporabnikom te organizacije zavrne, ter prisilno odjavo ali ponastavitev gesla posameznega uporabnika. Vsako spremembo, ki jo administrator izvede, zaledje zabeleži v tabelo `admin_audit_logs` skupaj s stanjem zapisa pred spremembo in po njej, kar omogoča poznejšo sledljivost posegov, zahtevano v razdelku 3.1.



Slika 5.5: Pregled organizacij v administratorski aplikaciji.

Poglavje 6

Evalvacija

V tem poglavju preverimo hipotezi H2 in H3 iz razdelka 1.2 tako, da razvito rešitev preizkusimo s testnimi uporabniki in njihovo oceno zaznane uporabnosti ovrednotimo s standardiziranim vprašalnikom System Usability Scale (SUS) [3], ki smo ga izbrali med sorodnimi vprašalniki.

6.1 Udeleženci in postopek

Za testiranje smo povabili 12 testnih uporabnikov izmed kolegov, družinskih članov in prijateljev, katerih demografsko in poklicno ozadje ustreza ciljni skupini aplikacije (zaposleni v manjših podjetjih, vodje manjših timov). Povprečna starost udeležencev je znašala 27,0 let (standardni odklon 9,7 leta), med njimi so bile 3 ženske in 9 moških. Vsak udeleženec je v vlogi zaposlenega, vodje ali kadrovske službe opravil kratek nabor nalog, ki pokrivajo osrednje module sistema, opisane v poglavju 5:

1. prijava v sistem in beleženje dogodka delovnega časa (prihod, odmor, odhod) v mobilni aplikaciji,
2. oddaja zahtevka za dopust in, v vlogi vodje ali kadrovske službe, odobritev zahtevka drugega udeleženca,

3. v vlogi vodje: samodejno generiranje osnutka urnika za teden dni, pregled predlaganih dodelitev in objava urnika,
4. prevzem odprte izmene v vlogi zaposlenega.

Po zaključenih nalogah je vsak udeleženec izpolnil vprašalnik SUS, predstavljen v razdelku 6.2. Testiranje je potekalo samostojno na lastni napravi, izvedba naštetih nalog (brez izpolnjevanja vprašalnika) pa je v povprečju trajala 10 minut.

6.2 Vprašalnik SUS

Vprašalnik SUS sestavlja deset trditev, na katere udeleženec odgovori s pet-stopenjsko lestvico strinjanja, pri čemer 1 pomeni “sploh se ne strinjam” in 5 “popolnoma se strinjam”. Trditve smo prevedli v slovenščino:

1. Mislim, da bi to aplikacijo rad(-a) uporabljal(-a) pogosto.
2. Aplikacija se mi je zdela po nepotrebnem zapletena.
3. Zdelo se mi je, da je aplikacijo enostavno uporabljati.
4. Mislim, da bi za uporabo te aplikacije potreboval(-a) pomoč tehnične osebe.
5. Zdelo se mi je, da so različne funkcije aplikacije dobro povezane med seboj.
6. Zdelo se mi je, da je v aplikaciji preveč neskladnosti.
7. Domnevam, da bi se večina ljudi zelo hitro naučila uporabljati to aplikacijo.
8. Aplikacija se mi je zdela zelo okorna za uporabo.
9. Pri uporabi aplikacije sem se počutil(-a) samozavestno.

10. Preden sem lahko začel(-a) uporabljati aplikacijo, sem se moral(-a) naučiti veliko stvari.

Trditve z lihimi zaporednimi številkami so pozitivno naravnane (višja ocena pomeni boljši rezultat), s sodimi pa negativno (višja ocena pomeni slabši rezultat). Posamezno oceno SUS izračunamo tako, da pri lihih trditvah od odgovora odštejemo 1, pri sodih pa odgovor odštejemo od 5, dobljene vrednosti seštejemo in zmnožimo z 2,5, kar da končno oceno na lestvici od 0 do 100. Bangor in sodelavci [2] so oceni SUS dodali pridevniško lestvico, katere mejne vrednosti je potrdil tudi Hertzum [7]: povprečna ocena 85 ustreza pridevniku “odlično”, 71 “dobro”, 51 pa “ok”, torej približno povprečju. Za razvrščanje rezultatov v nadaljevanju te meje poenostavimo takole: oceno nad 85 uvrščamo med “odlično”, med 71 in 85 med “dobro”, okoli 51 predstavlja povprečje, pod 51 pa velja za slabo.

6.3 Rezultati

Tabela 6.1 povzame oceno SUS za vsakega izmed 12 udeležencev.

Tabela 6.1: Individualne in povprečna ocena SUS.

Udeleženec	Ocena SUS
U1	90,0
U2	92,5
U3	100,0
U4	82,5
U5	92,5
U6	97,5
U7	85,0
U8	90,0
U9	67,5
U10	97,5
U11	70,0
U12	92,5
Povprečje	88,1

Povprečna ocena SUS znaša 88,1, kar po pridevniški lestvici sodi v razred “odlično”. Devet od dvanajstih udeležencev doseže oziroma preseže mejo 85 (“odlično”), en udeleženec, z oceno 82,5, pade v razred “dobro”, preostala dva pa v razred okoli povprečja (51), nobena ocena pa ne pade pod mejo povprečja ali v razred “slabo”. Take rezultate lahko pripišemo predvsem preprostosti nabora testnih nalog.

Glede na dobljene rezultate hipotezo H3, da bodo uporabniki zaznalo uporabnost aplikacije ocenili kot dobro, kar se bo pokazalo v povprečnem rezultatu SUS nad 71 točk, potrdimo. K hipotezi H2 o časovnem prihranku pri samodejnem razporejanju ne moremo podati enakovredno trdne trditve: v končni obliki vprašalnika ločenega vprašanja o prihranku časa v primerjavi z

ročnim razporejanjem nismo vključili, zato nimamo neposredne kvantitativne potrditve. Posredno jo podpira podatek, da so udeleženci nalogo generiranja, pregleda in objave osnutka urnika med testiranjem opravili hitro in brez večjih zapletov, kar je skladno tudi z visoko oceno trditve o povezanosti funkcij aplikacije (trditve 5). Hipotezo H2 zato lahko na podlagi izvedene evalvacije potrdimo le delno. Za trdnejšo potrditev bi bilo treba, kot navajamo v razdelku 7, prihranek časa izmeriti neposredno.

Poglavje 7

Zaključek

V diplomskem delu smo zasnovali in razvili spletno in mobilno aplikacijo za upravljanje delovnega časa in kadrovskih procesov, ki v enotnem sistemu združuje evidenco delovnega časa, razporejanje izmen z možnostjo samodejnega generiranja osnutka urnika in upravljanje odsotnosti, ob tem pa podpira dostop na osnovi vlog za administratorja, kadrovske službe, vodjo in zaposlenega ter nadzorni vmesnik za upravljanje več organizacij na eni platformi. Dosegli smo vse cilje, opredeljene v razdelku 1.1: zajeli smo zahteve po vlogah, zasnovali podatkovni model in arhitekturo (poglavje 3), razvito rešitev opisali po modulih (poglavje 5) in jo ovrednotili s testnimi uporabniki (poglavje 6).

Glede na rezultate evalvacije hipotezo H3, da bodo uporabniki aplikacijo ocenili kot uporabno in enostavno za uporabo, potrdimo: povprečna ocena SUS na vzorcu 12 testnih uporabnikov je znašala 88,1, kar po pridevniški lestvici Bangorja in sodelavcev [2] sodi v razred “odlično”. Hipotezo H2 o časovnem prihranku pri samodejnem razporejanju lahko na podlagi izvedene evalvacije potrdimo le delno, saj časovnega prihranka v primerjavi z ročnim razporejanjem nismo izmerili neposredno, temveč jo posredno podpira hitra in nemotena izvedba naloge razporejanja med testiranjem. Hipotezo H1, da enoten sistem zmanjša potrebo po ročnem usklajevanju podatkov med ločenimi orodji, neposredno nismo merili s testnimi uporabniki, temveč jo utemeljujemo z zasnovano samega sistema (poglavje 3), v katerem so evidenca delovnega

časa, razporejanje izmen in odsotnosti od začetka povezani na skupnem podatkovnem modelu, ne da bi bilo med njimi potrebno ročno prenašanje podatkov. K podobnemu sklepu vodijo tudi raziskave s področja digitalizacije kadrovskih procesov: Mahmoud in sodelavci [9] na vzorcu zaposlenih pokažejo, da digitalni kadrovske informacijske sistemi statistično značilno zmanjšajo zapletenost nalog in potrebo po ročnem delu ter s tem izboljšajo učinkovitost kadrovske funkcije.

7.1 Omejitve rešitve

Razvita rešitev ima kljub doseženim ciljem več omejitev. Reševalnik za samodejno razporejanje je požrešna heuristika, ki izmene dodeljuje kronološko in ne poišče nujno globalno najboljše razporeditve za celotno obdobje, kot smo pojasnili v razdelkih 2.2 in 5.4. Pri zahtevnejših urnikih z veliko omejitvami bi eksaktni ali metaheuristični pristop lahko dal boljši rezultat, a na račun daljšega časa izvajanja in zahtevnejšega vzdrževanja kode. Evalvacija je potekala na majhnem, priložnostnem vzorcu udeležencev (kolegi, družina, prijatelji), zato dobljenih ocen SUS ne moremo posplošiti na širšo populacijo uporabnikov kadrovskih sistemov, vendar pa je potrebno povedati, da so udeleženci sodili v ciljno skupino uporabnikov. Sistem prav tako še ne podpira obračuna plač niti integracije z zunanjimi kadrovskimi ali računovodskimi sistemi, ki jih ponujajo nekatere komercialne rešitve iz razdelka 2.1.

7.2 Nadaljnje delo

Rešitev bi lahko nadgradili v več smereh. Reševalnik razporejanja bi lahko razširili z možnostjo, da zaposleni za posamezno izmeno oddajo eksplicitno ponudbo namesto zgolj splošne preference, kar bi vodilo do močnejši signal pri odločanju, ali pa reševalnik nadomestili z metaheurističnim oziroma eksaktnim pristopom, na primer s celoštevilskim programiranjem, ki bi lahko poleg zasedenosti optimiziral tudi strošek dela. Smiselna razširitev bi bila tudi

podpora obračunu plač na podlagi evidentiranega delovnega časa. Evalvacijo bi bilo koristno ponoviti na večjem in bolj raznolikem vzorcu uporabnikov, morda v obliki daljšega uporabniškega preizkusa v realnem podjetju, kar bi omogočilo tudi merjenje dejanskega prihranka časa pri razporejanju izmen, ne le subjektivne ocene uporabnikov.

Literatura

- [1] BambooHR LLC. *BambooHR*. URL: <https://www.bamboohr.com> (pridobljeno 10. 7. 2026).
- [2] Aaron Bangor, Philip Kortum in James Miller. „Determining what individual SUS scores mean: Adding an adjective rating scale“. V: *Journal of Usability Studies* 4.3 (2009), str. 114–123.
- [3] John Brooke. „SUS: A Quick and Dirty Usability Scale“. V: *Usability Evaluation in Industry* (1996), str. 189–194.
- [4] Deputy. *Deputy*. URL: <https://www.deputy.com> (pridobljeno 10. 7. 2026).
- [5] Andreas T. Ernst in sod. „Staff scheduling and rostering: A review of applications, methods and models“. V: *European Journal of Operational Research* 153.1 (2004), str. 3–27. DOI: 10.1016/S0377-2217(03)00095-X.
- [6] Kraig Finstad. „The usability metric for user experience“. V: *Interacting with Computers* 22.5 (2010), str. 323–327. DOI: 10.1016/j.intcom.2010.04.004.
- [7] Morten Hertzum. „System Usability Scale: A Meta-Analysis of How SUS Relates to Workload, Task Time, and Error Rate“. V: *International Journal of Human-Computer Interaction* (2026). Published online 16. 2. 2026. DOI: 10.1080/10447318.2026.2625260.
- [8] Xin Liu in sod. „A fast-flexible strategy based approach to solving employee scheduling problem considering soft work time“. V: *Scientific Reports* 14 (2024). DOI: 10.1038/s41598-024-56745-4.

-
- [9] Mo'men Hani Mahmoud in sod. „The impact of digital HRM system and digital transformation on HR efficiency with organizational agility as a moderator“. V: *Discover Sustainability* 6.1 (2025), str. 1038. DOI: 10.1007/s43621-025-01713-9.
- [10] Personio SE & Co. KG. *Personio*. URL: <https://www.personio.com> (pridobljeno 10. 7. 2026).
- [11] Ravi S. Sandhu in sod. „Role-based access control models“. V: *IEEE Computer* 29.2 (1996), str. 38–47. DOI: 10.1109/2.485845.
- [12] Jorne Van den Bergh in sod. „Personnel scheduling: A literature review“. V: *European Journal of Operational Research* 226.3 (2013), str. 367–385. DOI: 10.1016/j.ejor.2012.11.029.
- [13] When I Work, Inc. *When I Work*. URL: <https://wheniwork.com> (pridobljeno 10. 7. 2026).